EXAMEN PARCIAL 4

Algoritmos y Estructuras de Datos

Fecha: ----------

Modelo de Práctica - Enunciado:

Un entrenador Pokémon profesional necesita organizar y analizar la información de los pokémons que tiene a su disposición para determinar cuáles son los más adecuados para distintos tipos de combate. De cada **Pokemon** se conoce: nombre (cadena de caracteres), número en la Pokédex (entero positivo, entre 1 y 1025), tipo (entero entre 1 y 18, donde: 1-Acero, 2-Agua, 3-Bicho, 4-Dragón, 5-Eléctrico, 6-Fantasma, 7-Fuego, 8-Hada, 9-Hielo, 10-Lucha, 11-Normal, 12-Planta, 13-Psíquico, 14-Roca, 15-Siniestro, 16-Tierra, 17-Veneno, 18-Volador), puntos de combate (PC) (entero positivo), y nivel de evolución (entero entre 1 y 6, donde: 1-Sin evolución, 2-Evolución inicial, 3-Evolución secundaria, 4-Evolución final, 5-Mega-evolución, 6-Regresión primigenia).

En base a lo anterior, se provee **ya desarrollado**:

- ✓ Un módulo *clases.py* que contiene la definición de la clase **Pokemon** con el *método constructor* y el método *__str__()* ya incluidos. NO CAMBIE NADA. NO AGREGUE NADA. DEJE LA CLASE TAL CUAL ESTÁ.
- ✓ Un segundo módulo *principal.py* que contiene una función *main()* con el esquema del menú de opciones ya desarrollado y funcionando, de forma que esa función *main()* sea usada como la función de entrada del programa completo y una función de validación de carga. En el mismo módulo, se provee ya desarrollada una función *generar_aleatorio()* que crea con valores aleatorios un objeto de la clase **Pokemon** y lo retorna ya listo para usar. Los procesos para cumplir con cada opción del menú, son justamente la tarea que debe programar cada estudiante. Las opciones a programar son:

---

1. Cargar un arreglo de registros con los datos de n pokémons (cargar n por teclado y validar que sea positivo), de manera que en todo momento el arreglo se mantenga ordenado por *número en la Pokédex*. Para esto debe utilizar el algoritmo de inserción ordenada con búsqueda binaria (se considerará directamente incorrecta la solución basada en cargar el arreglo completo y ordenarlo al final, o aplicar el algoritmo de inserción ordenada, pero con búsqueda secuencial). La carga de los Pokémons debe ser automática y con base aleatoria (use la función *generar_aleatorio()* que ya tiene disponible en el módulo *principal.py* para crear los objetos). Cada vez que se ingrese en esta opción, el arreglo debe ser creado desde cero y todo contenido anterior debe ser eliminado.

   Ajuste de runner: al ejecutar esta opción, solo cargue por teclado el valor de n, y no muestre nada al terminar de cargar el vector. Solo regrese al menú.

---

2. Mostrar todos los pokémons del arreglo, a razón de un registro por línea. Contar (en un único contador) cuántos eran de alguno de tres tipos específicos t1, t2 y t3 (los tres son ingresados por teclado) y mostrar ese contador al final en una línea extra.

   Ajuste de runner: al ejecutar esta opción, cargue por teclado los valores de t1, t2 y t3, muestre el listado pedido usando el método __str__() que ya tiene en la clase, muestre al final el contador pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

---

3. Obtener la cantidad de Pokémons según el tipo de Pokémon y nivel de evolución. O sea, 18 x 6 contadores, en una matriz de conteo). Mostrar únicamente los contadores que sean mayores a cero.

   Ajuste de runner: al ejecutar esta opción, asegúrese de que la matriz sea de orden 18 x 6 (Y NO 6 x 18). Para mostrarla, recórrala por filas, muestre cada contador pedido indicando a qué tipo de Pokémon y a qué nivel de evolución corresponde cada uno (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

---

4. A partir del arreglo de registros, generar un archivo binario donde se incluyan todos los datos de los Pokémons que sean aptos para participar en combates con máximo de 1500 PC (puntos de combate).

   Ajuste de runner: al ejecutar esta opción, solo genere el archivo pedido y regrese al menú (no muestre

nada en la pantalla al terminar).

---

5. Mostrar el contenido del archivo binario, a razón de un registro por línea en la pantalla, indicando además una línea extra que muestra el promedio de puntos de combate de los pokémons de un tipo *t* que debe ser ingresado por teclado. Nota: para el cálculo del promedio, debe recorrer el mismo archivo, Y NO calcular el promedio desde el vector.

   Ajuste de runner: al ejecutar esta opción, cargue el valor t por teclado, muestre el listado pedido usando el método __str__() que tiene disponible en la clase, muestre al final el promedio pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

Criterios generales de evaluación.

- Estructura completa del programa terminado **[máximo: 4 puntos por convenciones de estilo, la estructura de las funciones que desarrolle, el control de ejecución del módulo principal, y todo otro elemento que aporte a la estructura del programa]**
- Desarrollo correcto del ítem 1: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 2: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 3: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 4: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 5: **[máximo: 4 puntos]**
- Para aprobar el parcial (con nota mínima de 4(cuatro), el alumno debe llegar a un *porcentaje de al menos 55% del puntaje máximo (eso es, un poco más de 13 puntos)*.

.